

Introduction

- In this poster, we will:
- Justify the use of a Rotating Wave Approximation Hamiltonian
 - Construct Quantum Logic Gates, starting with the Phase and Hadamard
 - Look at an application of Quantum Logic Gates, the Quantum Fourier Transform

We want to introduce the Pauli matrices (Barnett 2026b, p. 15), [Physical/Exact Hamiltonian](#) and [Rotating Wave Approximation \(RWA\)](#) (Barnett 2026b, p. 18) to obtain quantum gates.

Pauli matrices:

$$\sigma_X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \sigma_Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad \sigma_Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

Physical Hamiltonian (Barnett 2026b, p. 15):

$$\mathcal{H}_{exact}(t) = \frac{\epsilon}{2}\sigma_Z + \gamma\sigma_X \cos(\omega t), \quad \epsilon, \gamma, \omega \in \mathbb{R}_{\geq 0}$$

In order to obtain the RWA, "it is customary to work entirely within the frame rotating at ω ". (Barnett 2026b, p. 18)

Rotating Frame:

$$|\psi'(t)\rangle = e^{i\frac{\omega}{2}\sigma_Z}|\psi(t)\rangle, \quad t \in \mathbb{R}_{\geq 0}$$

ROT/RWA:

$$\mathcal{H}_{rot}(t) = \frac{\epsilon - \omega}{2}\sigma_Z + \frac{\gamma}{2}\sigma_X + V(t), \quad \mathcal{H}_{RWA} = \frac{\epsilon - \omega}{2}\sigma_Z + \frac{\gamma}{2}\sigma_X$$
$$V(t) = \frac{\gamma}{2}(\sigma_X \cos(2\omega t) - \sigma_Y \sin(2\omega t))$$

We must use the Time-dependent Schrödinger Equation (Barnett 2026b, p. 9) to find a $|\psi(t)\rangle$:

$$\hbar \frac{\partial}{\partial t} |\psi(t)\rangle = \mathcal{H}(t) |\psi(t)\rangle$$

$\hbar = 1$ for all calculations in this poster.

Is the RWA viable?

Is the time-independent $\mathcal{H}_{RWA}$ solution a close approximation to the time-dependent $\mathcal{H}_{rot}(t)$ solution when we drop $V(t)$?

We obtain the following solutions to the Time-dependent Schrödinger Equation:

$$|\psi_{RWA}(t)\rangle = |\psi_{RWA}(0)\rangle e^{-i\mathcal{H}_{RWA}t}, \quad |\psi_{rot}(t)\rangle = \mathcal{T}exp(-i \int_0^t \mathcal{H}_{rot}(t')dt')|\psi_{rot}(0)\rangle$$

for an inner product, with $\mathcal{H}$ being picked as the RWA and Rotated Hamiltonian for each function, $\mathcal{T}$ is the time-ordering operator, derived from a Dyson series (Cappellaro 2012, p. 4).

We define an inner product function on Python and compute the time-averaged state [Fidelity](#) (how close 2 Quantum operations are to each other (Ivezic 2023)):

$$\bar{F} = \frac{1}{T} \int_0^T F(t) dt, \quad F(t) = |\langle \psi_{rot}(t) | \psi_{RWA}(t) \rangle|^2, \quad 0 \leq \bar{F} \leq 1, T \in \mathbb{R}_{\geq 0}$$

Deriving F(t) from Born Rule (Barnett 2026a, p. 11), the $\bar{F}$ is calculated for $T = 10$ using the Trapezium rule, using 10,000 points and a NumPy array (Max. Fidelity Error = 1 - min(Fidelity)).

Table 1: Fidelity and Max Fidelity Error (M.F.E). Obtained using NumPy (Harris et al. 2020) and SciPy (Virtanen et al. 2020) (trapezoid function).				
ϵ	γ	ω	Fidelity	Max. Fidelity Error
11	1	10	0.9980863	0.0066193
41	1	40	0.9998697	0.0004489
1001	1	1000	0.9999995	1.245E-6

When $\sqrt{(\epsilon - \omega)^2 + \gamma^2} \ll \omega$ is obeyed (which is required for negligible oscillations (Barnett 2026b, p. 17)), the solutions to the Schrödinger Equation are extremely similar, so we can drop $V(t)$.

Realised Single Qubit Gates

Using our Rotating Wave Approximation, we must obtain realised gates of the form $U(\tau) = e^{-i\mathcal{H}_{RWA}\tau}$ (Barnett 2026b, p. 20) and compare U to the sought form of a quantum gate. What parameters do we substitute in order to realise the sought quantum gates?

Table 2: Realised Gates				
Gate	Sought Form	γ	τ	ϵ
<i>Phase</i>	$e^{-i\phi\sigma_Z}$	0	$\frac{2\phi}{\epsilon - \omega}$	$\ll \omega$
<i>Hadamard</i>	$\frac{1}{\sqrt{2}}(\sigma_X + \sigma_Z)$	$\epsilon - \omega$	$\frac{\pi}{\sqrt{2}\gamma}$	$\ll \omega$
<i>X/NOT</i>	σ_X	$\neq 0$	$\frac{(2k+1)\pi}{\gamma}$	ω
$\sqrt{X}$	$\sqrt{\sigma_X}$	$\neq 0$	$\frac{(2k+1)\pi}{2\gamma}$	ω
$\frac{\pi}{8}/T$	$e^{-i\sigma_Z\frac{\pi}{4}}$	0	$\frac{\pi}{2(\epsilon - \omega)}$	$\ll \omega$
<i>S</i>	$e^{-i\sigma_Z\frac{\pi}{2}}$	0	$\frac{\pi}{\epsilon - \omega}$	$\ll \omega$
$R_X(\theta)$	$e^{-\frac{i\sigma_X\theta}{2}}$	$\neq 0$	$\frac{\theta}{\gamma}$	ω
$R_Y(\theta)$	$e^{-\frac{i\sigma_Y\theta}{2}}$	$\neq 0$	$\frac{\theta}{\gamma}$	ω

These are correct up to global phase, which is physically irrelevant.

$R_X(\theta), R_Y(\theta)$ (derived via identity) are the Rotation gates with a respective axis X and Y. $\theta \in \mathbb{R}$

$U(\theta, \phi, \lambda)$ is the General Single Qubit Rotation Gate defined as $R_Z(\phi)R_Y(\theta)R_Z(\lambda)$ ($\phi, \theta, \lambda \in \mathbb{R}$). Can be derived using the ideas of rotations. (Nielsen and Chuang 2010, p. 174-177)

Picking $\phi = \frac{\theta}{2}$ in the Phase gate yields $R_Z(\theta)$.

$k \in \mathbb{Z}$ for the X/NOT and $\sqrt{X}$ gates.

It is also very important to know the Dyadic Rational Phase Gate (Nielsen and Chuang 2010, p. 218):

$$R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{\frac{2\pi i}{2^k}} \end{pmatrix}$$

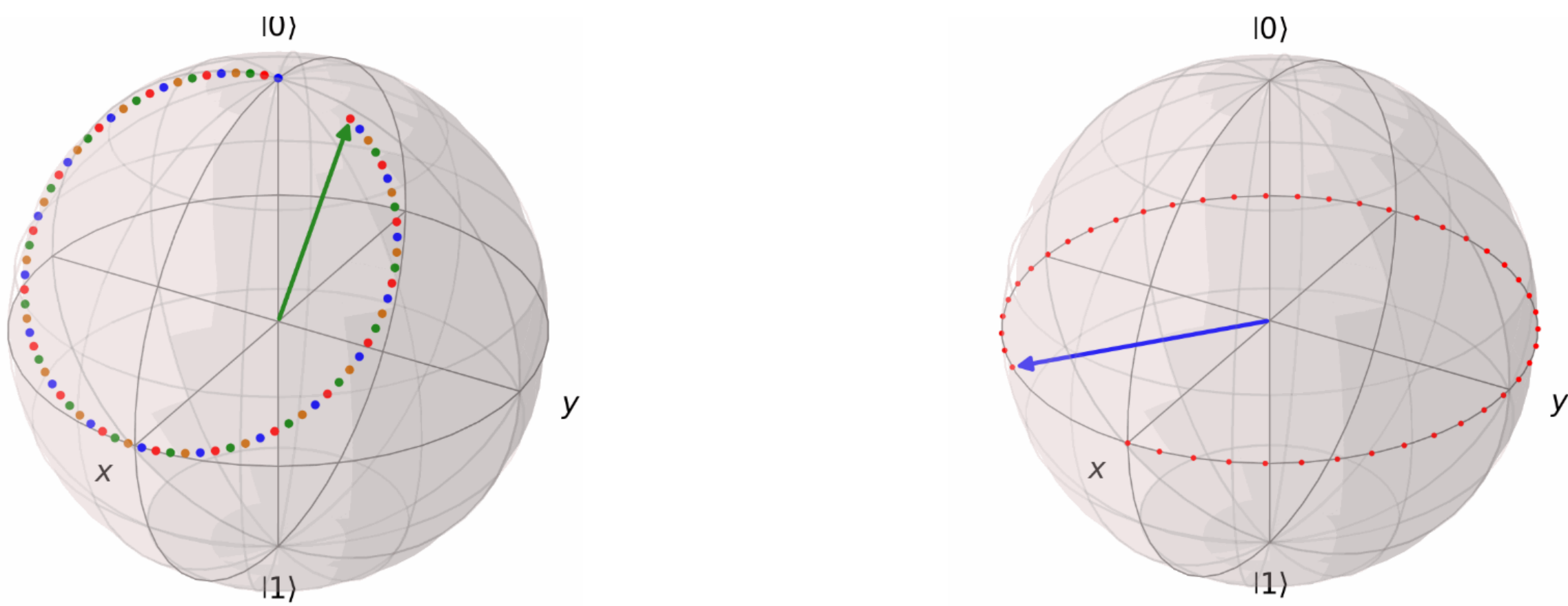
We later use this gate for Quantum Fourier Transforms, with $R_1 = Z, R_2 = S, R_3 = \frac{\pi}{8}/T$ gate. ($k \in \mathbb{N}$)

The Bloch Sphere

It is helpful to be able to visualise these quantum gates, and a useful way of doing this is the [Bloch Sphere](#). To get the idea:

- σ_X, σ_Y and σ_Z are π radians anticlockwise generators of rotations, up to a global phase, with exponentiation causing continuous rotation (e.g. $R_X(\pi) \equiv X$ as $R_X(\pi) = -iX$).
- The Phase gate is an anticlockwise rotation about the Z axis, angle ϕ .

Below, the Hadamard gate has been applied twice on $|0\rangle$ and the Phase gate has been applied on $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. It can be seen below that $H^2 = I$.



These are produced as GIFs using Python's QuTip (Lambert et al. 2026), NumPy (Harris et al. 2020), ImageIO (ImageIO Contributors 2026) and OS packages.

Quantum Fourier Transform

Now we want to introduce the 2-qubit SWAP gate by name, which will be important in the application of the [Quantum Fourier Transform \(QFT\)](#).

We define the QFT by using the Discrete Fourier Transform. The QFT (Nielsen and Chuang 2010, p. 217) definition is seen below:

$$|j\rangle \mapsto \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{\frac{2\pi i j k}{N}} |k\rangle$$

Where N : Length of state, $|k\rangle$: a basis of transformed data, $|j\rangle$: a complex state of input data.

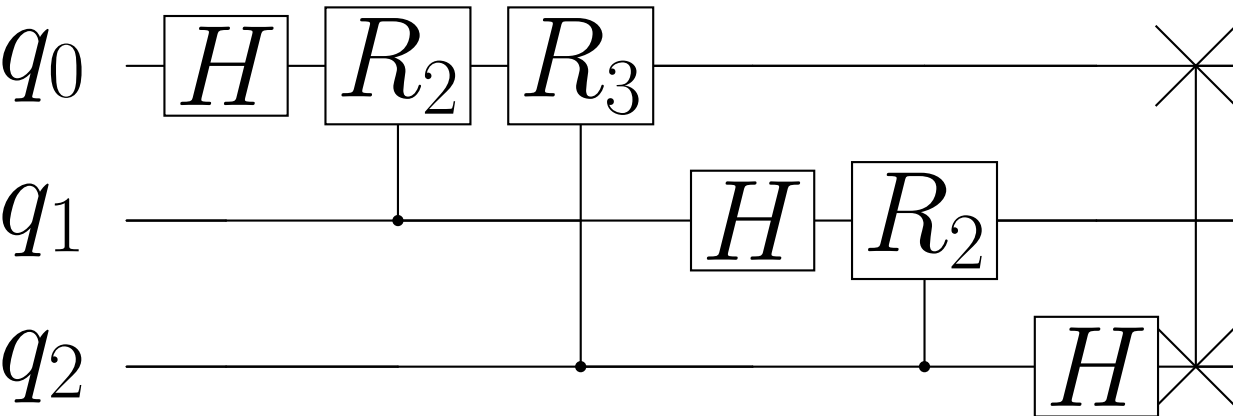
When computing a Quantum Fourier Transform, we want to take $N = 2^n$ where n is the number of qubits in a Quantum Computer.

Writing $|j\rangle = |j_1, \dots, j_n\rangle$ in a binary representation, where $j = j_1j_2\dots j_n$, we can eventually derive a product expression (Nielsen and Chuang 2010, p. 218) for the QFT using tensor operations:

$$|j\rangle \mapsto \frac{(|0\rangle + e^{2\pi i 0.j_n})(|0\rangle + e^{2\pi i 0.j_{n-1}.j_n}|1\rangle)\dots(|0\rangle + e^{2\pi i 0.j_1j_2\dots j_n})}{2^{\frac{n}{2}}}$$

Where we use the notation $0.j_1j_{1+1}\dots j_m = \frac{j_1}{2} + \frac{j_{2+1}}{4} + \dots + \frac{j_m}{2^{m+1}}$ which represents a binary fraction.

To see what this visually looks like for $n = 3$ (a 3-qubit QFT), we present the following circuit:



Note that the initial state is represented as $|q_0q_1q_2\rangle$. This figure was created using Qiskit (Javadi-Abhari et al. 2024) and verified using NVIDIA's CUDA-Q v13.2 (NVIDIA 2026), applying a comparison against the QFT matrix that we apply on $|000\rangle$ to obtain our expected result (Qiskit and CUDA-Q are Python packages).

We can see the SWAP gate being used at the end of the QFT, where at most $\frac{n}{2}$ SWAP gates are required for a given QFT applied on n qubits (Nielsen and Chuang 2010, p. 220).

We can compare the QFT (on n qubits) to the classical algorithm, the Fast Fourier Transform (FFT) (on N samples) (Nielsen and Chuang 2010, p. 220):

$$[QFT, \quad \Theta(n^2)] \quad [FFT, \quad \Theta(N \log_2 N)], \quad N = 2^n$$

From this we can see that the QFT is much faster at achieving a Discrete Fourier Transform in its respective usage when compared to the classical FFT.

If I had more time, I would consider the following:

- Shor's Algorithm, $O((\log(N))^3)$, for factorising integers (Nielsen and Chuang 2010, p. 37).
- Grover's Algorithm, $O(\sqrt{N})$, for unordered searches of N elements.
- Limitations of the QFT (hardware, availability)

As per college guidelines, I acknowledge that I have used GenAI for the purpose of searching for literature (textbooks, Python packages), solving LaTeX issues (formatting, packages) and getting feedback on my work (including GitHub files).

Citations, Derivations and Python

